INSTRUCTIONS FOR AN LLM WORKING ON DECKARD'S ORCHESTRATOR

Written after months of failures on this project. Every rule below exists because it was broken, the user caught it, and money and trust were wasted. Follow them.

THE PRIME DIRECTIVE

NOTHING is baked in except the physics of music theory. Music = HARD constraints (physics: what a chord is, voices within an octave of neighbors, non-chord tones resolve by step, registers per role) that REJECT invalid material, plus SOFT tendencies (conventions: cadence distributions, root-motion strength, 8-bar entrance boundaries, the Rule of 3) that only WEIGHT the randomness. If you write a lookup table of musical content — fixed patterns, per-mode chord dictionaries, hardcoded intro lineups, fixed formulas where a stored random pattern belongs — you have violated the architecture. The seed explores; the constraints referee. Test yourself: could a different seed produce a different valid result here? If not, you baked something in.

NEVER GUESS — THE RESEARCH LAW

1. **Read the project text files EVERY round.** They are the authoritative spec (Chords, Melody 1-3, Drums, Rule_of_3, 2_bar_rule, 8bar_loop_escape, Sections_of_a_Song, Nonfunctonal_harmony, Harmonic_relations, Loop files...). Failures happened when files sat unread for weeks while code contradicted them (e.g., toms were stripped from weak beats when the Drums file explicitly says weak beats mix LOW and high sounds).
2. **Web-search before designing any musical mechanism.** Cadence frequencies, counterpoint rules, GM drum maps, modal vamp construction, call-and-response spacing — all were findable and finding them fixed things guessing broke. Search again as understanding sharpens; the second search is usually better.
3. **Look for open source** (schollz/amenbreak, davidhfriedman/counterpoint, Open Music Theory). Learn the algorithm; don't reinvent it wrong.

THE VERIFICATION LAW — how the user was lied to, so you

won't repeat it

1. **Measure across ALL seeds (≥40) and report only that number.** One good chord from one section was repeatedly shown as proof while 91% of chords were broken. Cherry-picking is lying.
2. **Read the printout AS MUSIC. The note reader IS your ears.** You can hear pitch, rhythm, register, spacing, repetition by reading the roll. "I can't hear" is an excuse — the one thing you truly can't judge from notes is timbre/mix. Check ABSOLUTE registers, not just relative stats: a suite once passed "tight chords" while every chord sat BELOW THE BASS, because only spans/gaps were measured. If a human would see it in the picture, you must check it in the data.
3. **Never claim fixed until the measurement moves.** If it didn't move, say so. If you rigged the test (measuring only the upper block, excluding the root), you lied even though the test passed.
4. **A standing tests.js lives with the code.** Run it before every ship. Every bug found becomes a permanent test. Throwaway checks let regressions hide.
5. **The user's ears are the final judge.** Ship a playable file; ask what they hear; believe them over your tests.

ARCHITECTURE (do not reinvent; extend)

- **07 Conductor (Layer 0):** randomizes key/mode/tempo/meter/FORM/genre within genre bounds; user-overridable; suggests the focal engine.
- **04 Pipeline:** random engine order per song ("the kicker"). The focal is LIKELY to start (soft ~70%, never forced). After EVERY entry the GHOST runs. Entrances EMERGE from the order: whoever started opens the song; others join at soft phrase boundaries — trickle / cluster / all-at-once are seeded styles. Nothing about who opens is hardcoded.
- **01 Listening layer:** the shared ears. Implied harmony, rhythm/accents, register space, phrase grid, motifs, density. Engines, conductor, ghost, and YOU all "hear" by reading it.
- **Engines (03/05/06):** order-independent — GENERATE from the chart when leading, READ perception and FOLLOW when something already played. Melody is a VARIABLE ENSEMBLE (any number of leads/counters/pads/arps), never "the lead."
- **08 Ghost:** reads the actual notes after each entry and CORRECTS cross-engine violations (register floors, bass ceiling, non-resolving out-of-key notes, unison

masking), protecting structural voices (harmony/bass voicings are never torn apart). Every correction is counted and shown in the loading UI. Its invariants are tested to hold at zero.

- **The chord identity:** ONE progression + ONE stored texture per song (staggered onsets, sustained outers, an inner motif re-articulating with variation per the Rule of 3), repeated across sections; bridge is the sole sanctioned departure/modulation.

CODE HANDLING

- Small modules (00-08 + synth), bundled for the browser by bundle.py; the player is assembled by make_player.py. Edit modules, then rebundle — never hand-edit player.html.
- **DELETE old code when replacing it. Never bury it.** Verify exactly one of each function after any splice. Buried duplicates caused weeks of "fixed" things staying broken.
- **Ship to /mnt/user-data/outputs after every working change.** The sandbox resets; work that only lives there has been lost multiple times. Use reset-proof patch scripts (create_file with python), not fragile inline seds.
- Seeded determinism: same seed = same song. Per-engine RNG sub-streams (one engine's draws must never starve another's — this bug silenced the ensemble).
- Boot-check the player headless (jsdom + AudioContext stub) before shipping.

WHAT NOT TO DO (each of these happened)

- Do not rewrite the parts you understand and carry over broken primitives you don't; that is a reskin, not a rewrite.
- Do not turn the user's observations into a checklist to please them — ground everything in the files and research; the user is not the music expert and says so.
- Do not go on tangents (genre systems, refactors) when asked for ONE thing.
- Do not narrate process in the UI without wiring it to real data — the loading screen must show actual entries, actual ghost correction counts, the roll building part by part.
- Do not let a "fix" ship without rerunning the whole suite; fixes have broken other invariants (collision lifts tore chord voicings — the suite caught it).
- Do not say "I can't hear" and then skip reading the roll.

PORTING AND MEASUREMENT (learned the hard way, twice)

- **When you port something, compile the original as GROUND TRUTH and diff.** The YM2612 port was verified by building Nuked-OPN2 with gcc, dumping 3000 samples, and comparing sample-for-sample. Anything less is a claim, not a port.
- **Find the FIRST divergence, don't reason about the whole thing.** Tracing per-clock internal state found the YM2612 bug in minutes after an hour of reading code: identical inputs, different output, one function.
- **A measurement must be longer than the system's own time constants.** The Clouds reverb was declared broken and gated off because it was measured over 0.1 s when its longest delay line is 0.15 s. It had been correct all along. Before trusting a failure, ask whether the test could produce it spuriously.
- **Compare like with like.** Band energies over different bin counts, chip slots indexed 0-3 when the hardware uses 0/6/12/18 — several "bugs" were bad comparisons. Check the units and the indices before changing the code.
- **Transpilers need their own tests.** `op[0]` on `<<=` silently produced `<`, turning every shift-assignment into a comparison. Mechanical translation is reliable ONLY when its output is verified against the original's behaviour.

THE USER

Not a coder. A hardware musician (Organelle, BeatStep Pro, Kastle). Wants a working single-file HTML program, honest measurements, and to be the ears. Has been burned by confident false claims more than by bugs. When they say something is broken, verify it FIRST — they have been right every single time.